Ekte Mat Online – Oblig 4 report

Oct 24, 2026 ·

Ekte Mat Online is the ordering website and staff admin panel for Ekte Mat AS, a fictional caterer in Bergen. The version tagged `Obligatorisk-4` has all nine required features (C1–C5, A1–A4), follows MVC with Flask, and passes 305 automated tests.

1. Front page

Item	Value
Course	INFO212 Systemutvikling, autumn 2026: Obligatory assignment 4
Group members	*Magnus Råve, Ida Fyllingen, Jone Kuven, Tord Eiken*
GitHub repository	github.com/Rave00/Ektemat-Oblig-4
Version to assess	Tag **`Obligatorisk-4`**: the release Obligatorisk 4 - Ekte Mat, commit `14caa3a`
Neo4j included	**No.** The data is stored in SQLite, and only the repository classes in the model layer touch it. A Neo4j version is planned for Oblig 5 (`docs/plans/6 - Neo4j-Storage.md`).

The README gives the commands to install, seed, run and test the application; every larger feature has an approved plan in `docs/plans/`.

2. Features

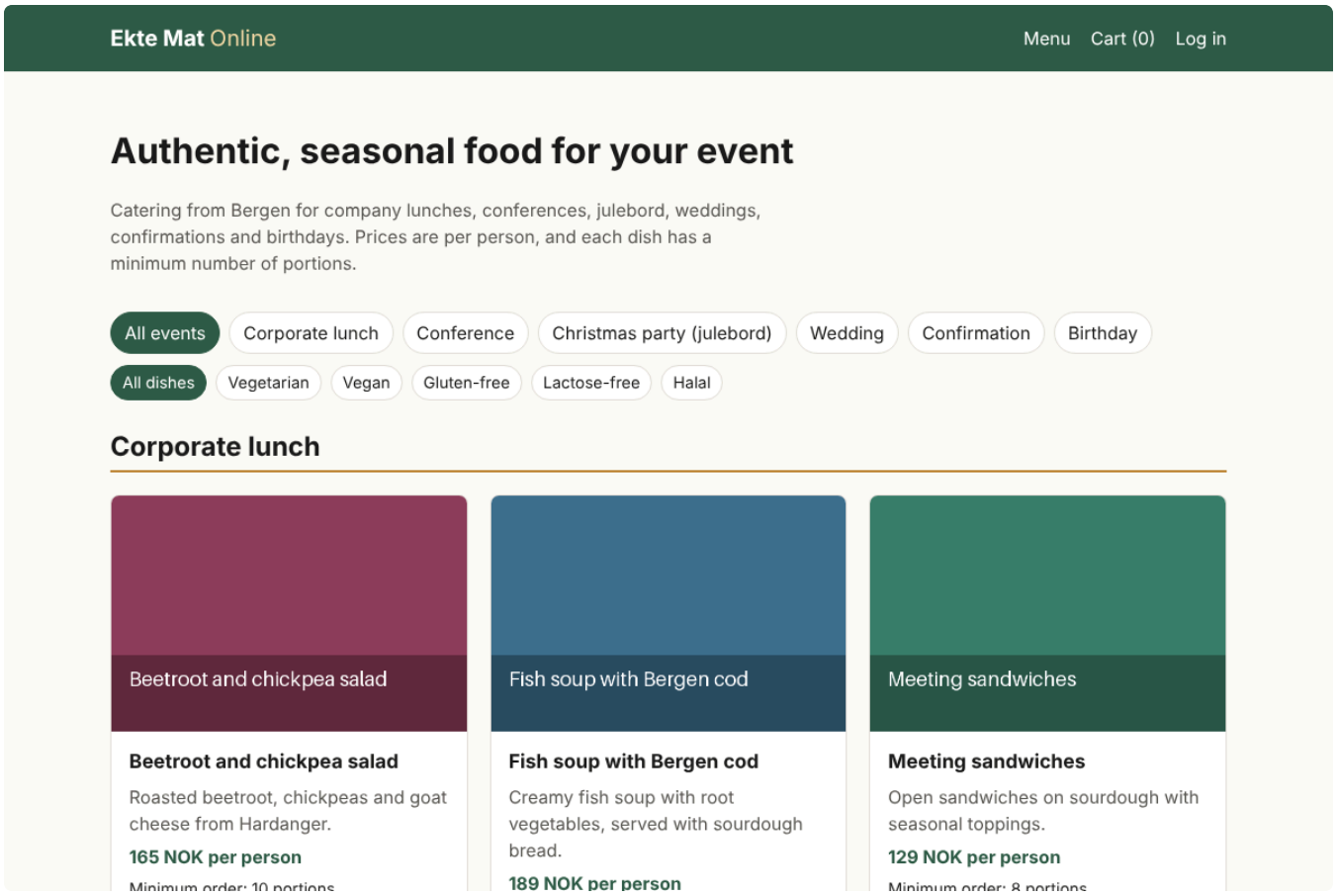
All nine required features are built and tested: five on the customer site (C1–C5) and four in the admin panel (A1–A4).

Feature	What it does	Requirement
Menu by event	Published dishes as cards grouped by event, each with picture, price per person, minimum order and allergens; filters by event and dietary label.	C1 · US-01, US-02
Shopping cart	Add, change and remove dishes; fewer portions than a dish's minimum are refused; estimated total in NOK.	C2
Register and order	Register or log in from the cart, then send event, date, guests, delivery address and allergy notes per guest group; the order page is the receipt.	C3 · US-09, US-40, US-47
My orders	The customer's orders with order status, payment status, total and questions.	C4 · US-11
Questions (customer)	Ask a question on an order and read Ekte Mat's answer on the same page.	C5 · US-21
Food items	Add, edit, publish and remove dishes; publishing is blocked without allergen information or a picture.	A1 · US-02, US-25
Images	Upload a picture, compare it with the current one, then save or discard it.	A2
Orders	All orders, filtered by status and event date; a page per order offers only the status changes allowed now, and sets the payment status.	A3 · US-17, US-19
Questions (staff)	The longest-waiting question first; each question is answered once, on the order page.	A4 · US-21

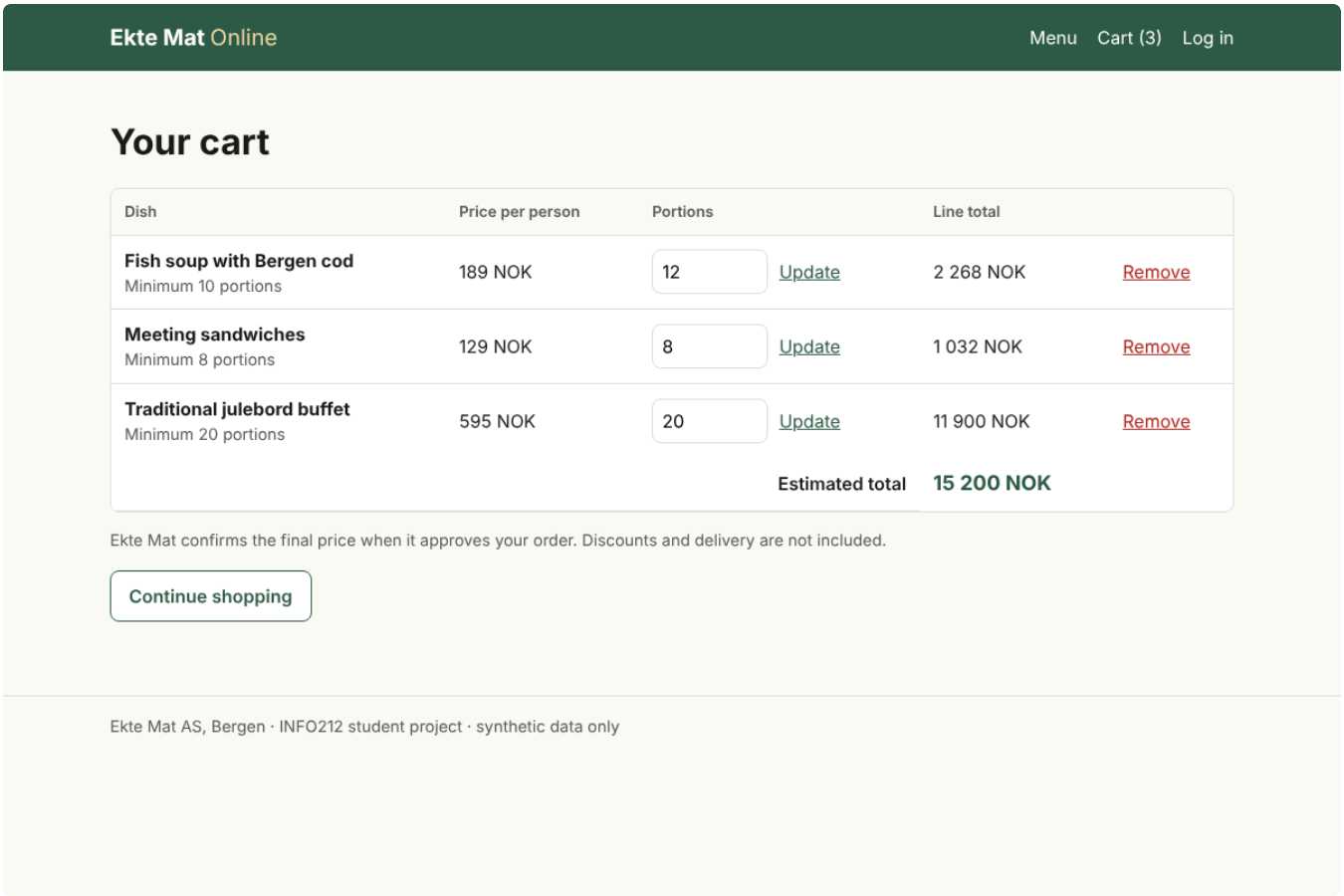
Beyond the minimum, each tied to a stakeholder need from the lab:

Feature	What it does	Stakeholder need
Dietary-label check	A dish cannot be saved as vegan with eggs or milk, vegetarian with fish, or gluten-free with gluten.	Customers choosing safe food for guests with allergies (US-02)
One order per click	A double click or a resent form stores one order, not two.	Duplicate orders in the old order data (V1)
Order data rules	1–400 guests, four-digit postcodes, no past event dates, and today's prices copied into each order.	Errors found in 820 rows of old orders (V4–V10)
Reply-time measurement	The dashboard shows, per month, the share of requests answered within 24 hours.	The owner's 95 % reply target, which nobody could measure (NFR-05, NFR-19)

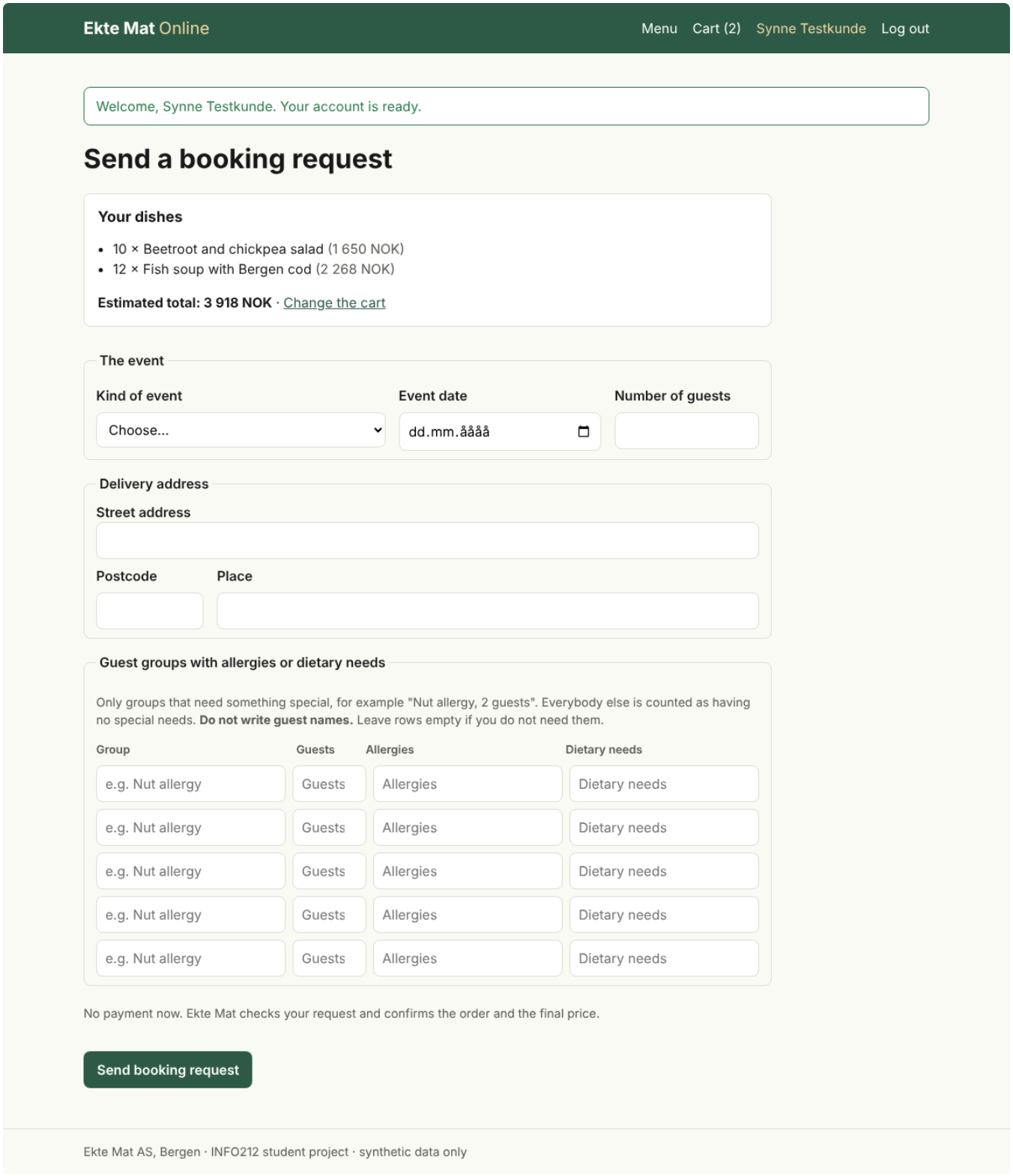
Customer site



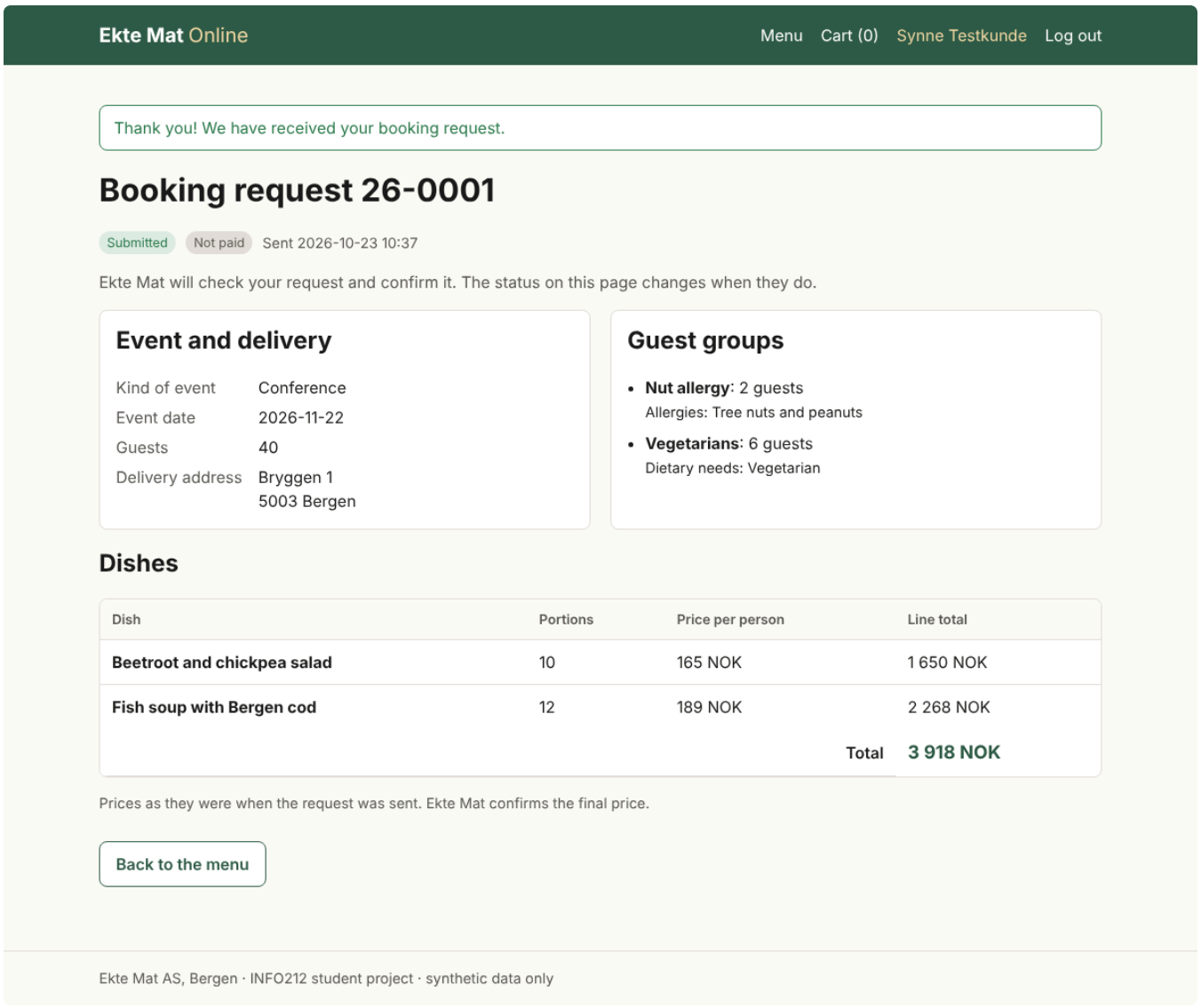
Menu (C1): dishes as cards grouped by event; the links filter by event and dietary label.



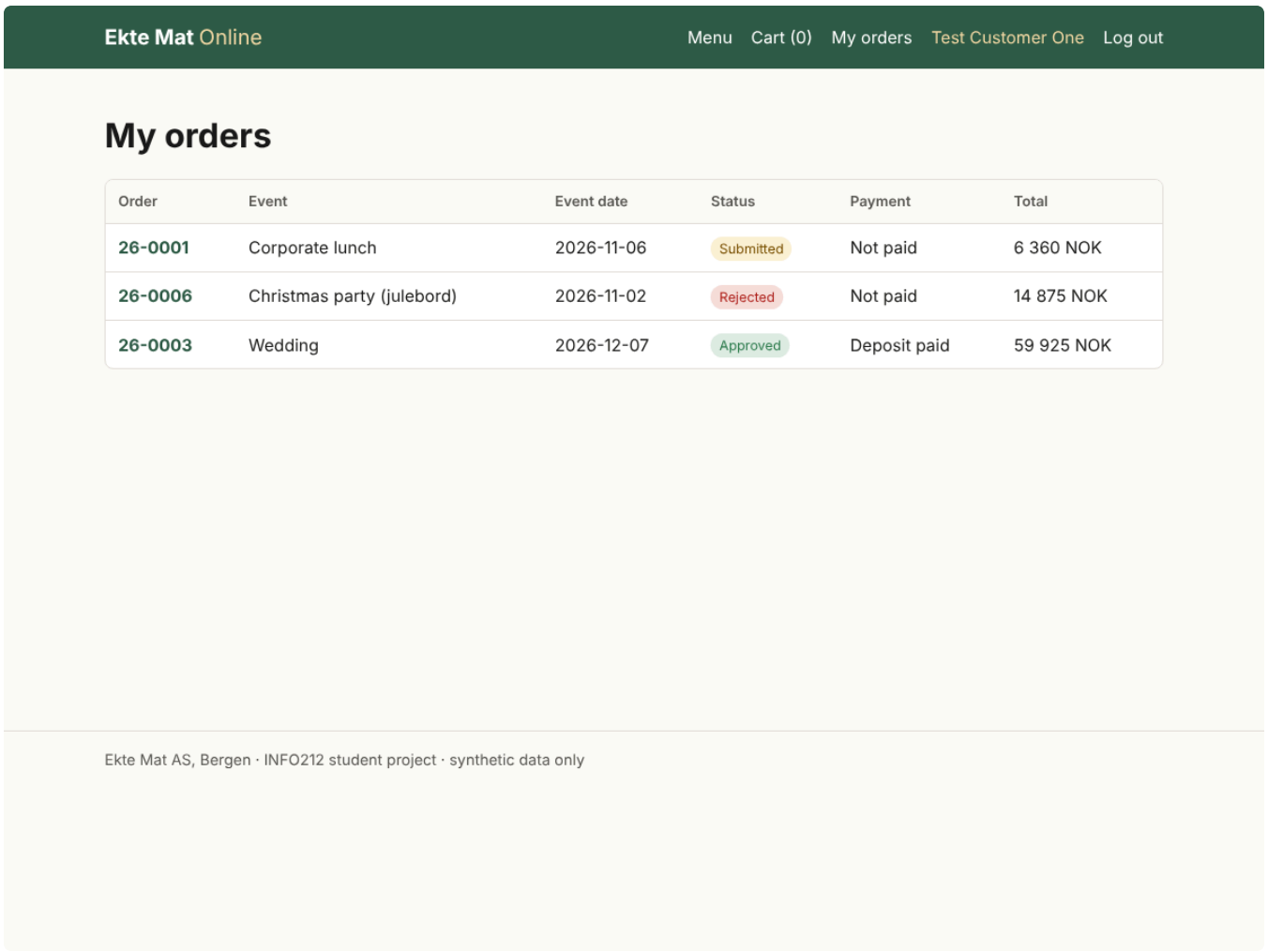
Cart (C2): portions, price per person and line totals; the estimated total in NOK.



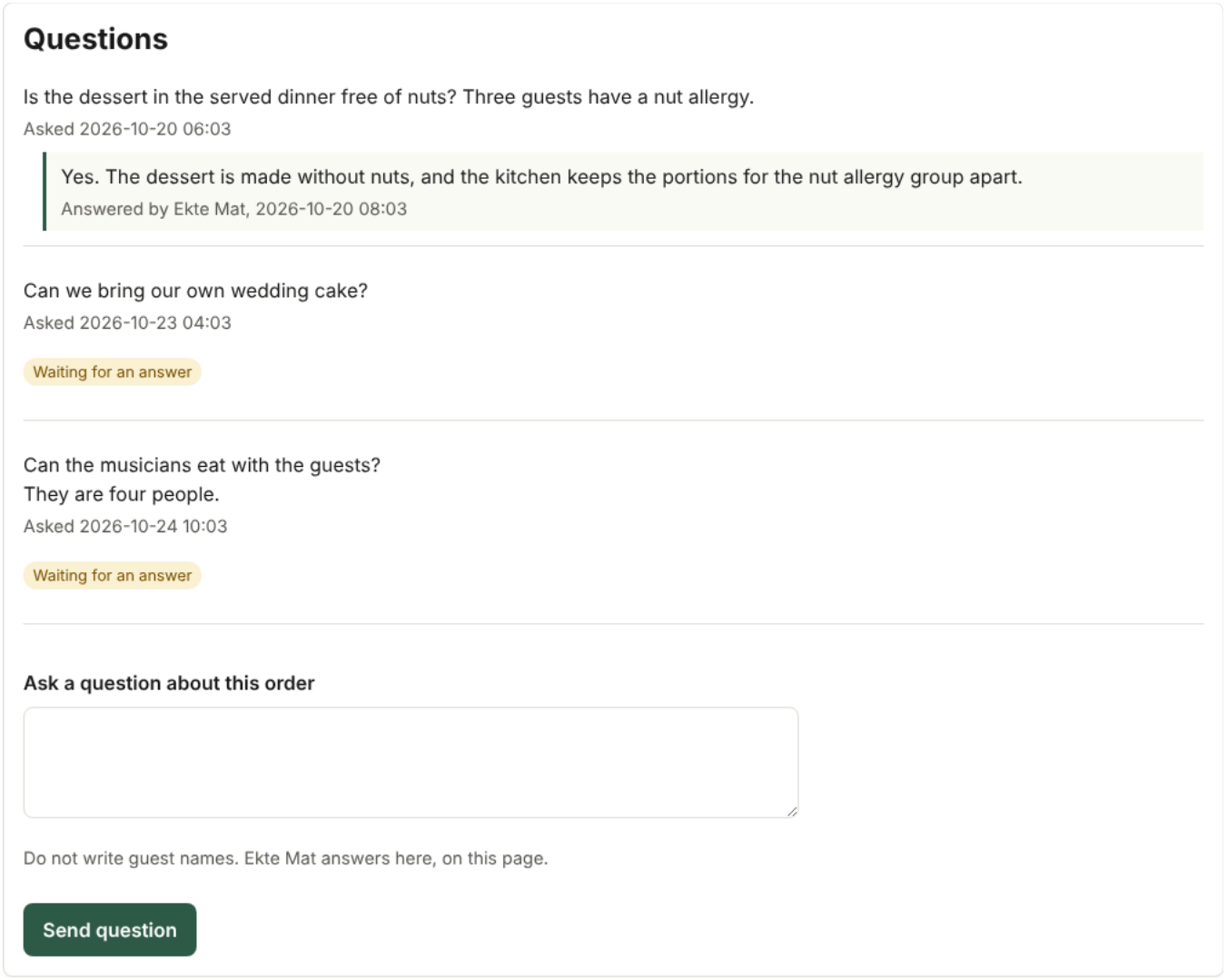
Order form (C3): event, date, guests, delivery address and up to five guest groups with allergies and dietary needs; no name fields.



Order page (C3): the receipt, with order number, status, guest groups and the prices copied into the order.

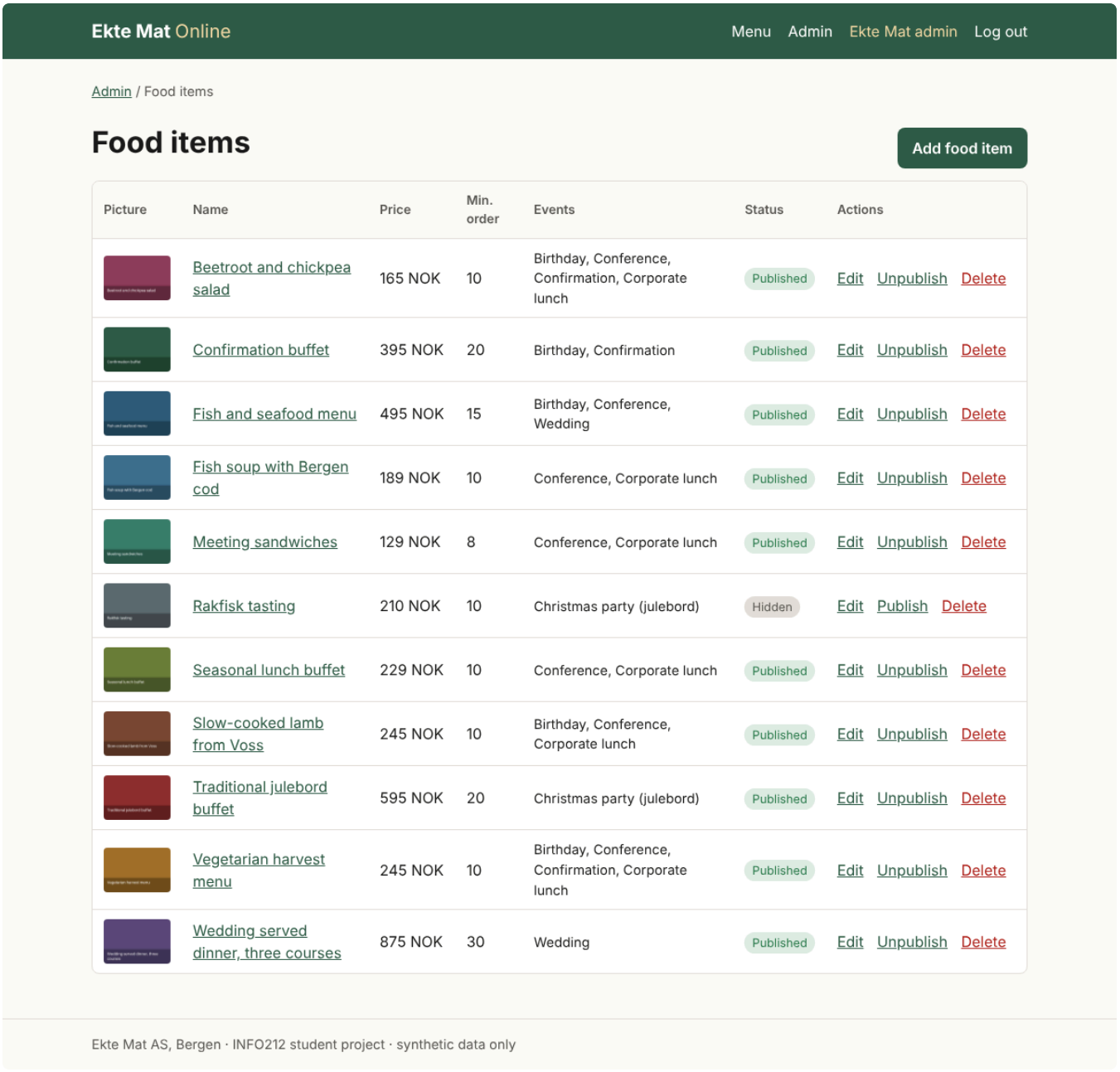


My orders (C4): each order with its order status, payment status and total.

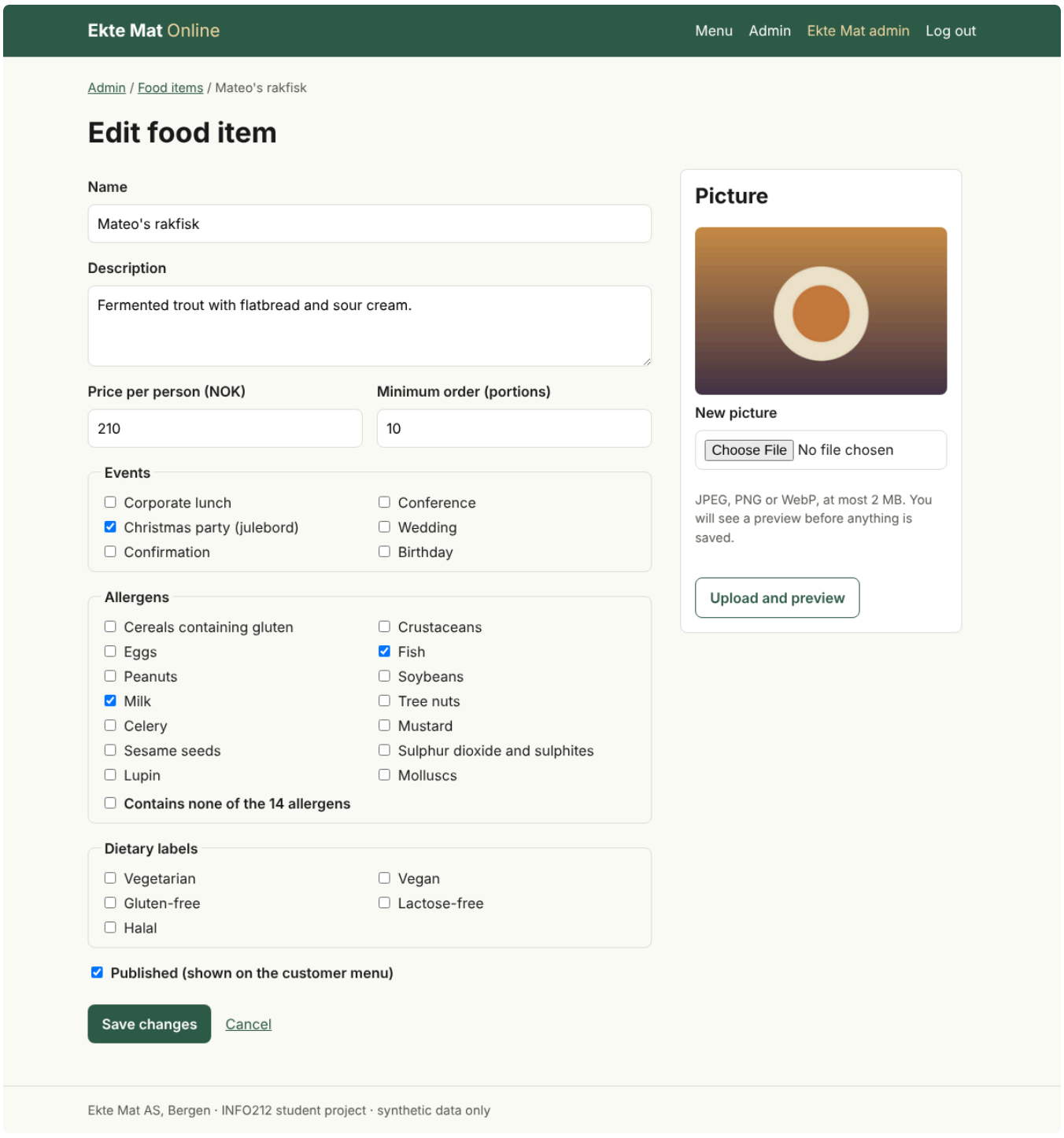


Questions (C5): one answered by Ekte Mat, two waiting, and the form to ask a new one.

Admin panel



Food items (A1): every dish with picture, price, minimum order, events and whether it is published.



Food item (A1): price, minimum order, events, the 14 allergens and dietary labels; the picture panel on the right.

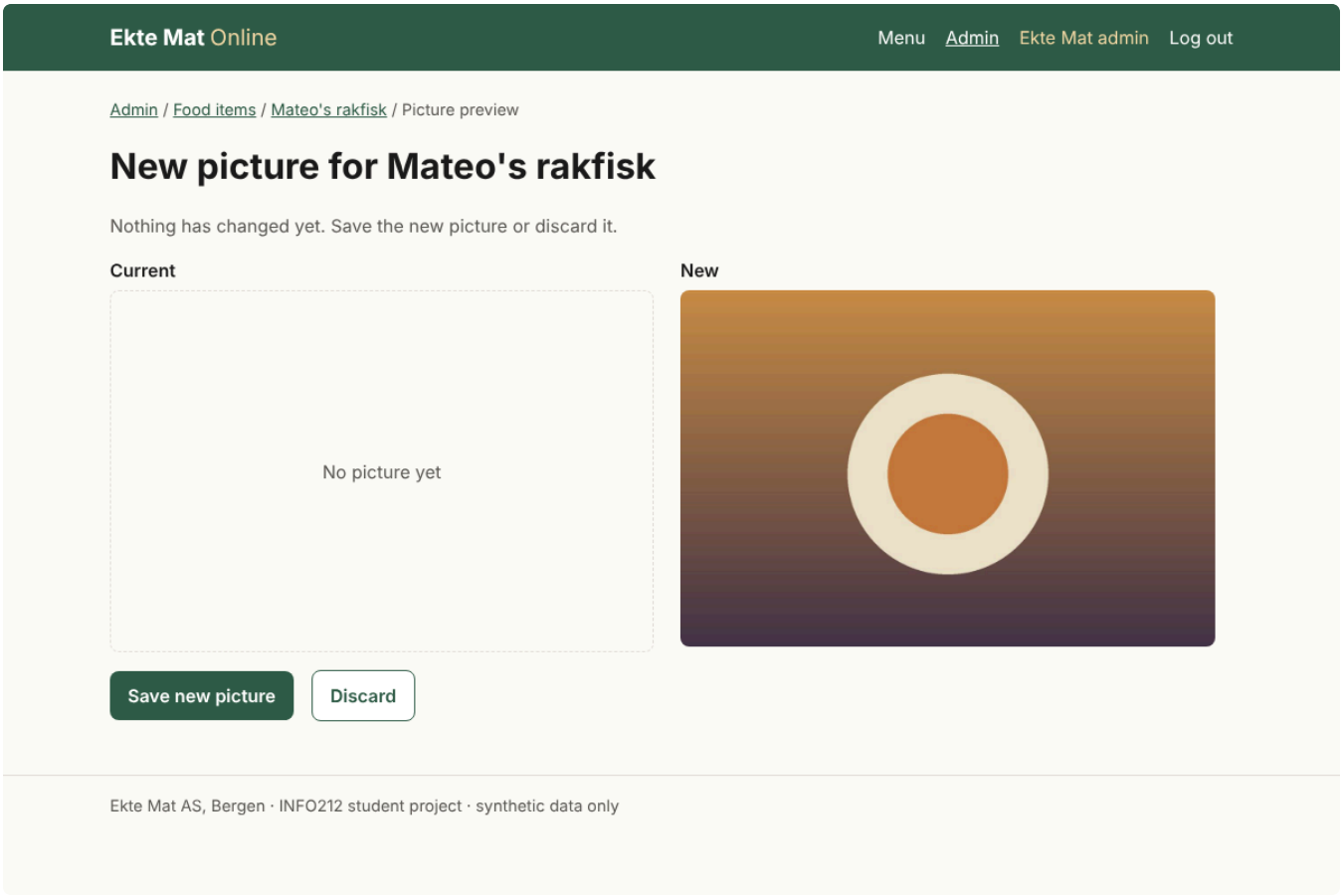
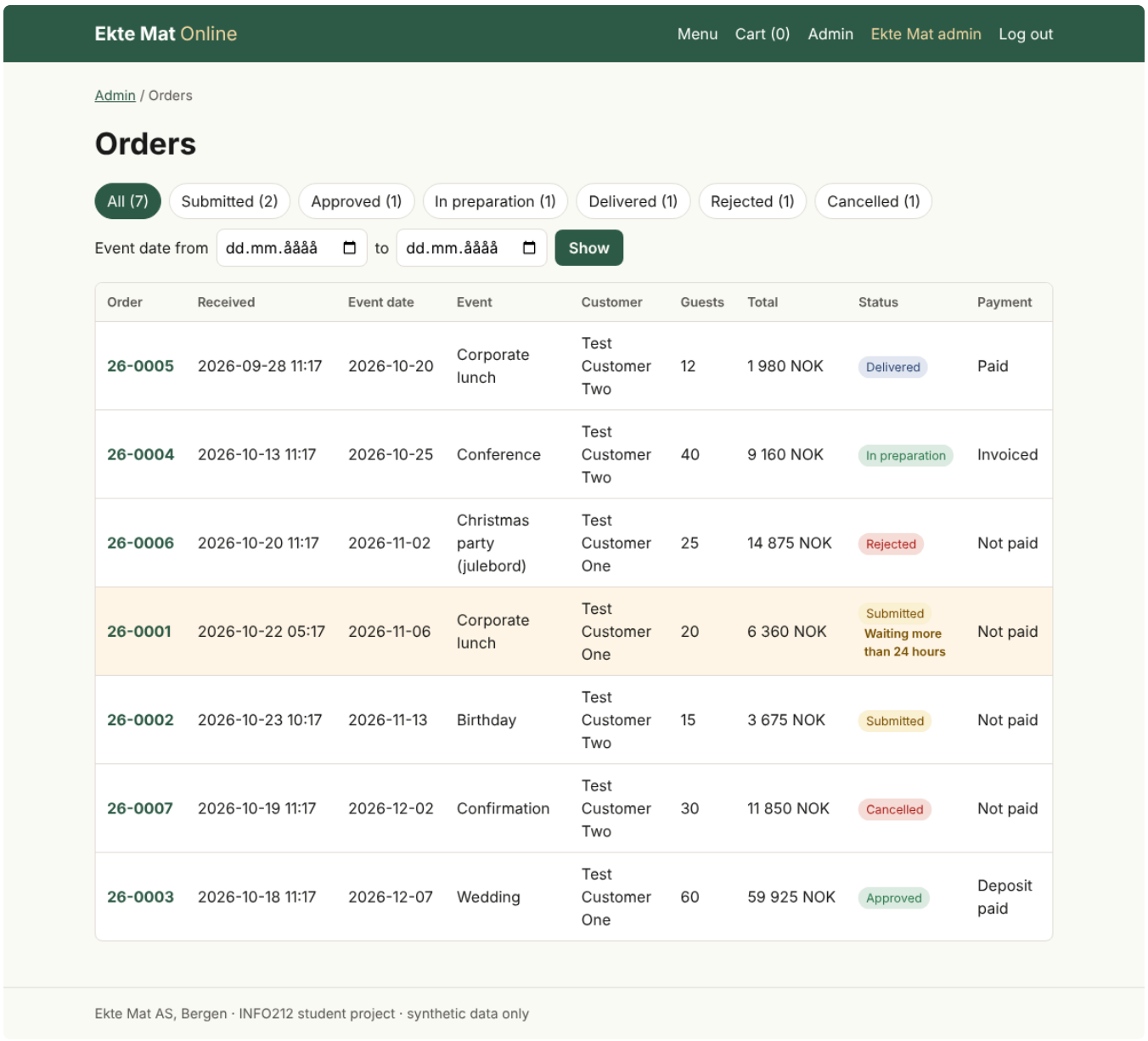
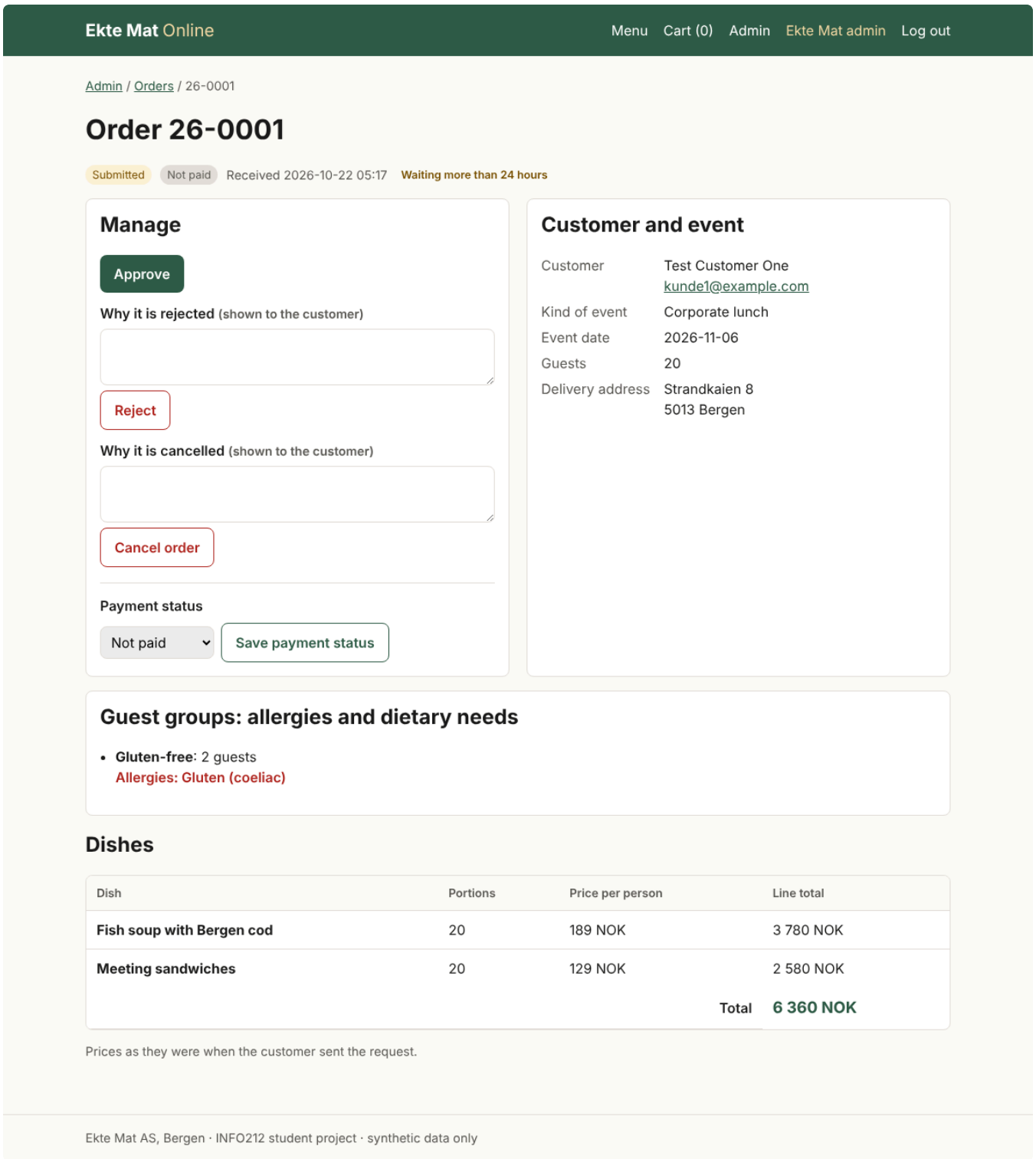


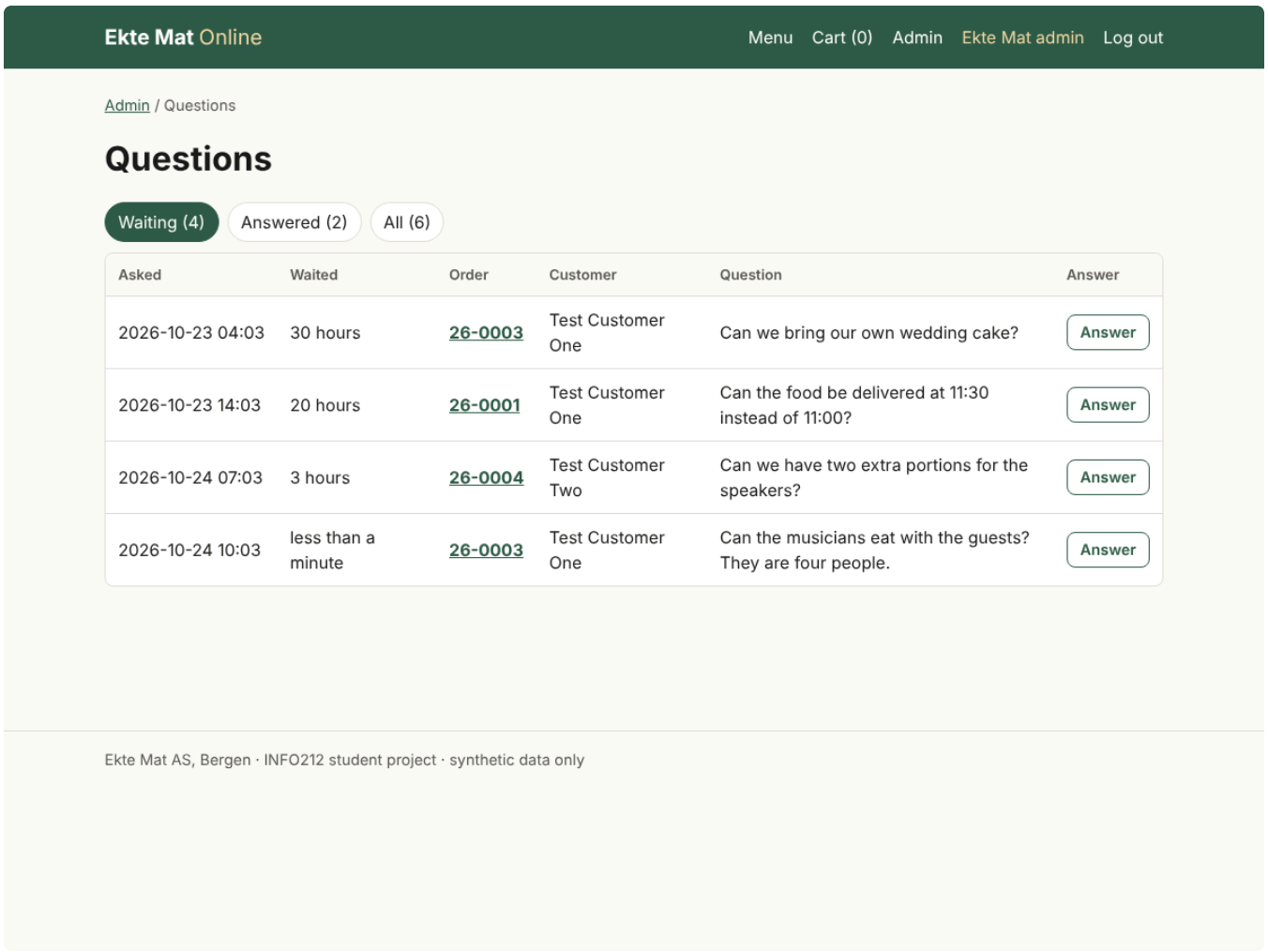
Image (A2): the new picture next to the current one; nothing changes until "Save new picture".



Order list (A3): sorted by event date, with status links and counts; an order waiting more than 24 hours is highlighted.



Order page (A3): only the status changes allowed now, a reason for the customer when rejecting or cancelling, the payment status, and the guest groups' allergies.



Questions (A4): those waiting for an answer, the longest-waiting first.

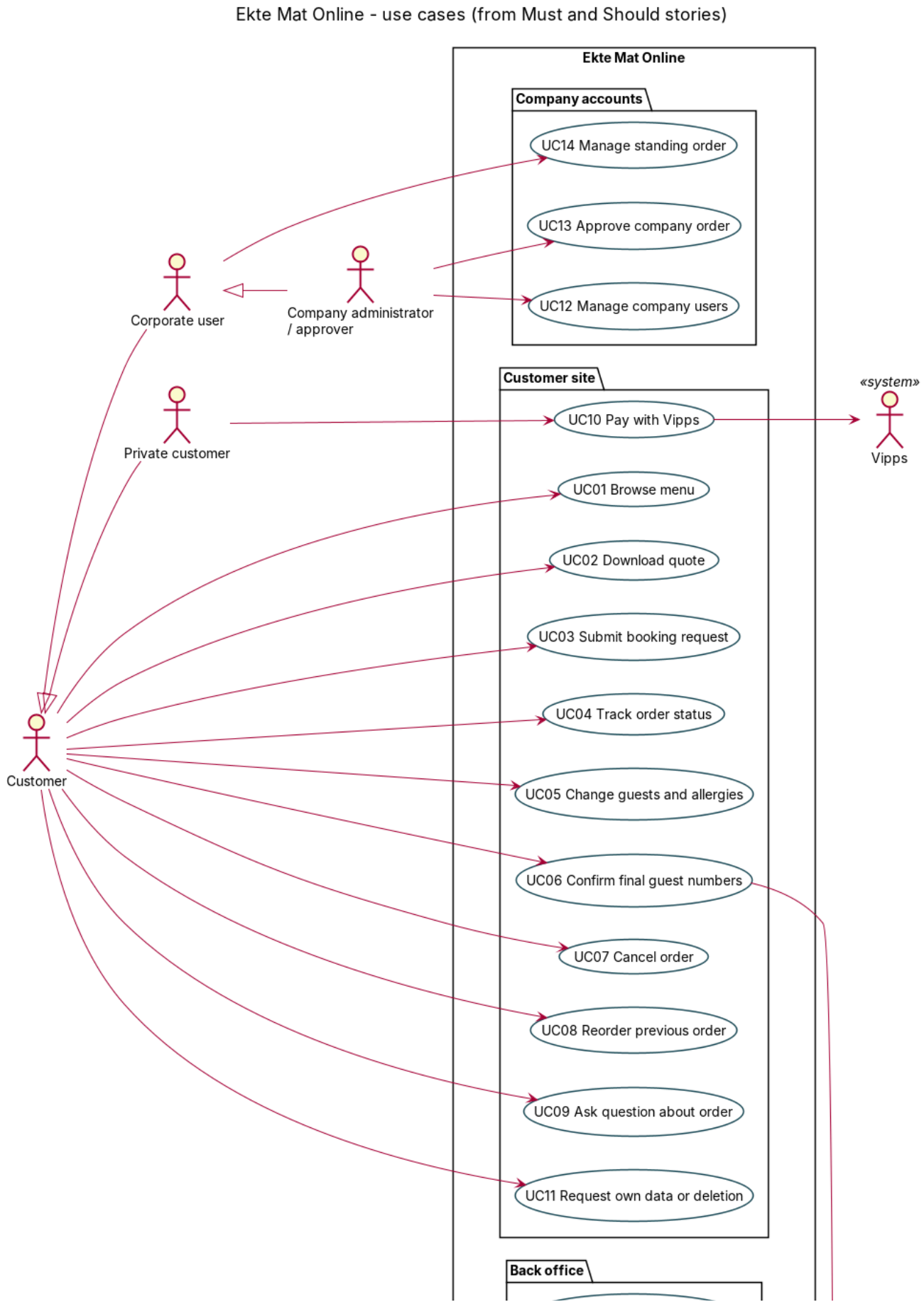
3. Software design

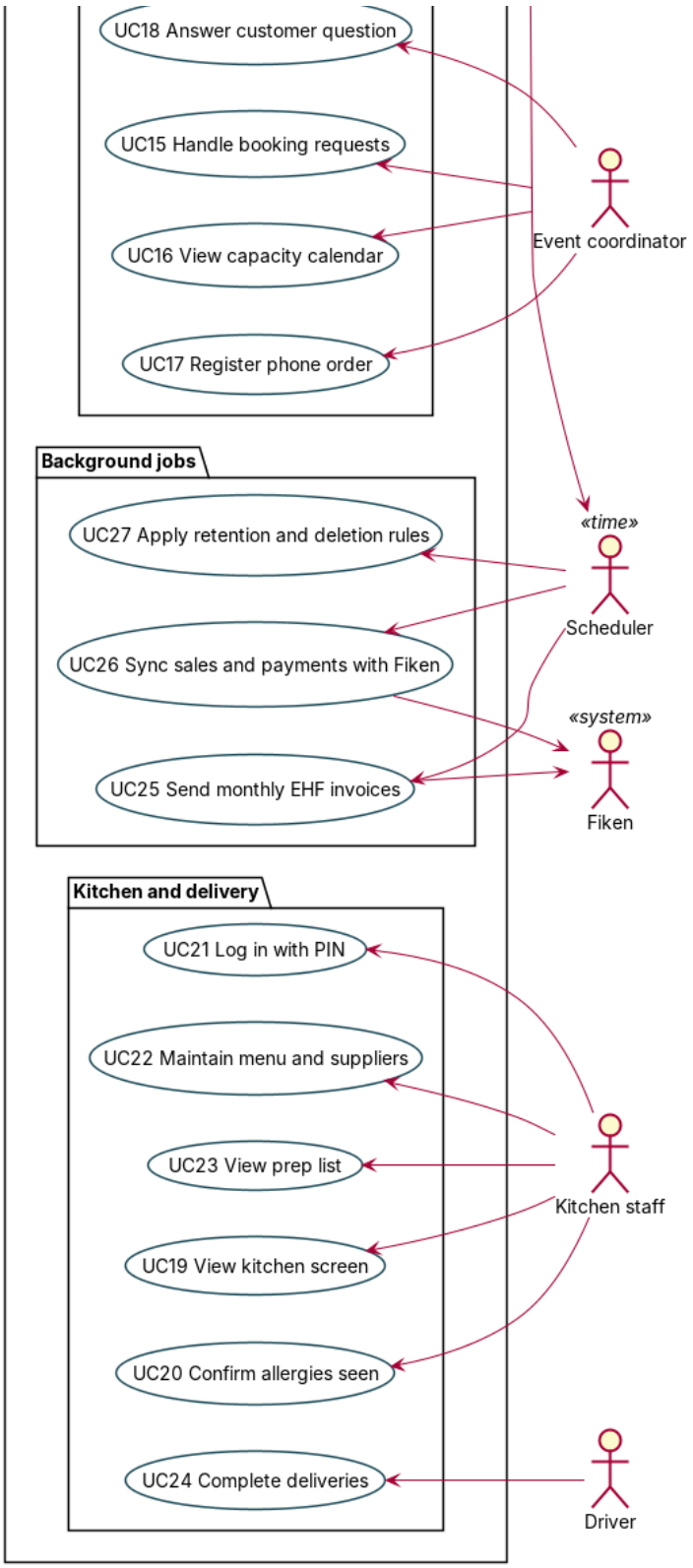
Eight design concepts shaped the code. The diagrams come from our lab analysis (`docs/design/`); the MVC drawing was made for this report.

User stories and acceptance criteria

The lab backlog (`docs/requirements/EkteMat_backlog_v3.xlsx`: 44 user stories, 22 non-functional requirements) was the input to every plan. Each plan names the stories it covers, and their Given/When/Then criteria became acceptance tests written before the code; US-02's message "Allergen information missing" is tested word for word. The app has 305 automated tests.

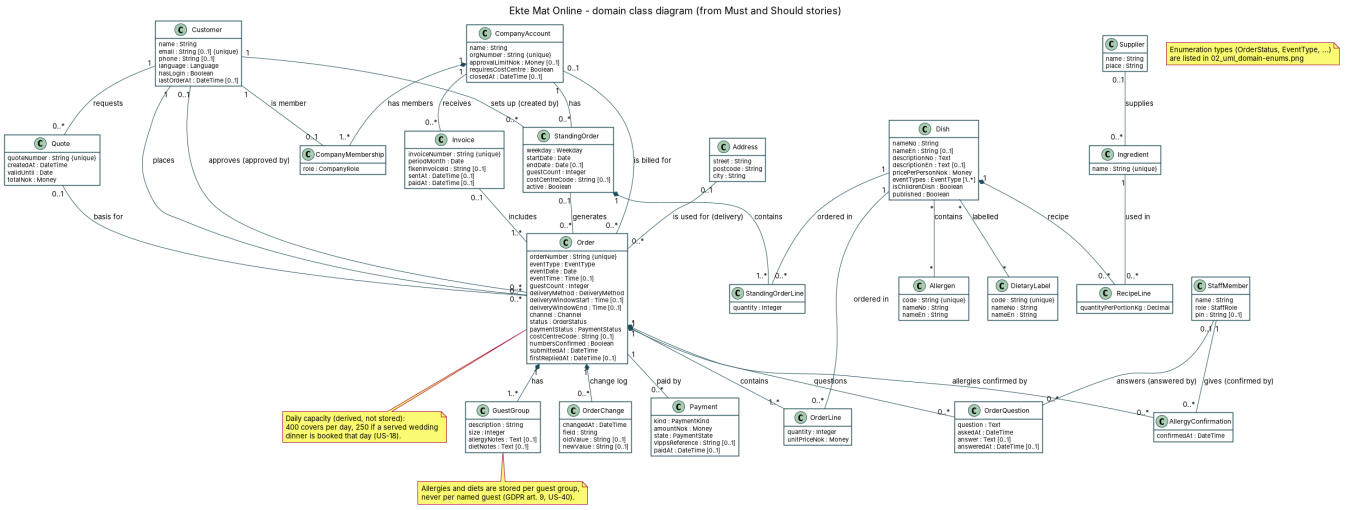
Use-case diagram


Ekte Mat Online - use cases (from Must and Should stories)
Ekte Mat Online
Company accounts
UC14 Manage standing order
UC13 Approve company order
UC12 Manage company users
Corporate user
Company administrator / approver
Private customer
Customer site
UC10 Pay with Vipps
«system»
Vipps
UC01 Browse menu
UC02 Download quote
UC03 Submit booking request
UC04 Track order status
Customer
UC05 Change guests and allergies
UC06 Confirm final guest numbers
UC07 Cancel order
UC08 Reorder previous order
UC09 Ask question about order
UC11 Request own data or deletion
Back office



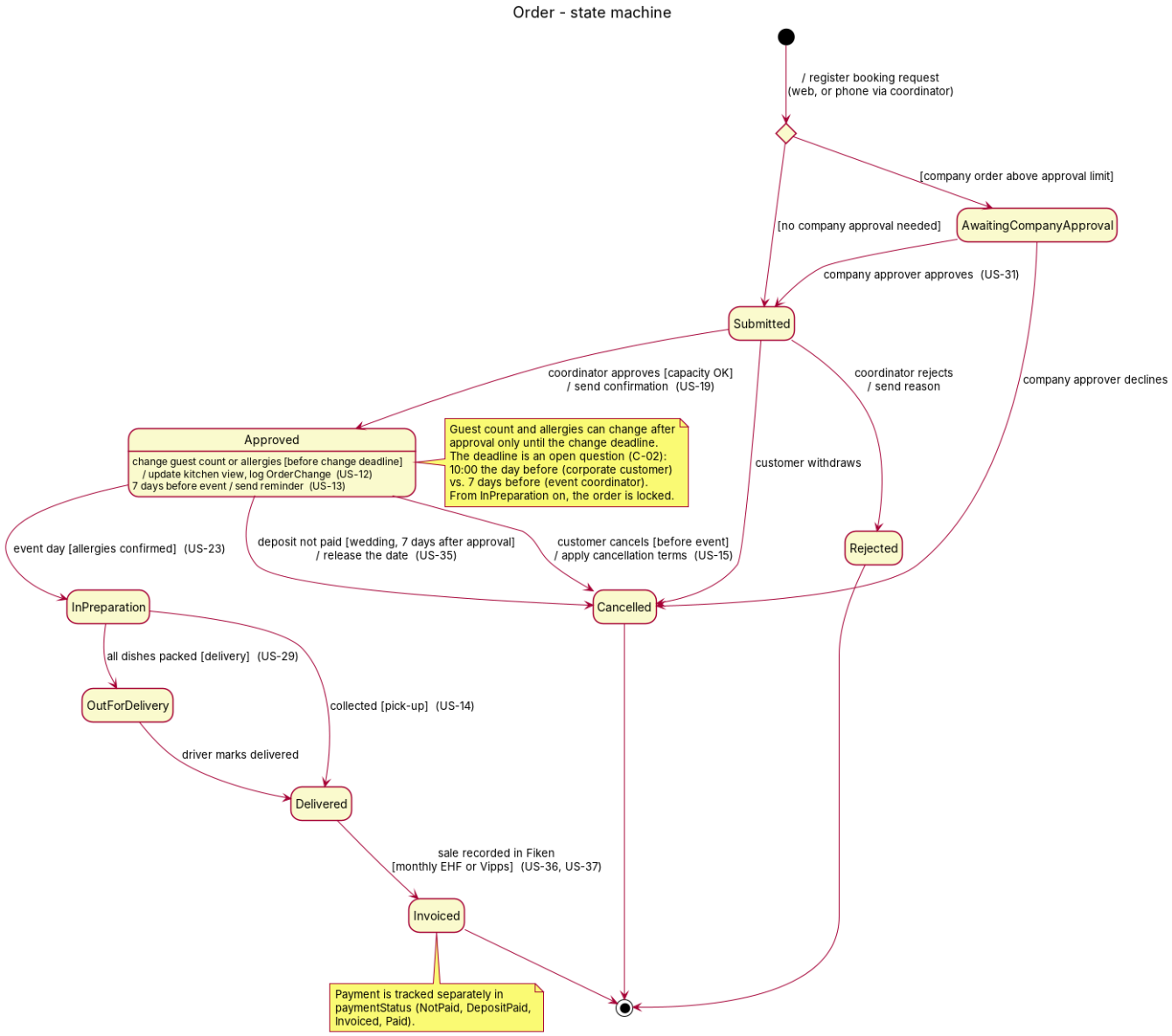
Used to set the scope and the order of the work. The customer's use cases (browse menu, submit booking request, track order status, ask a question about an order) became C1–C5; the staff's (maintain menu, handle booking requests, answer customer questions) became A1–A4.

Domain class diagram



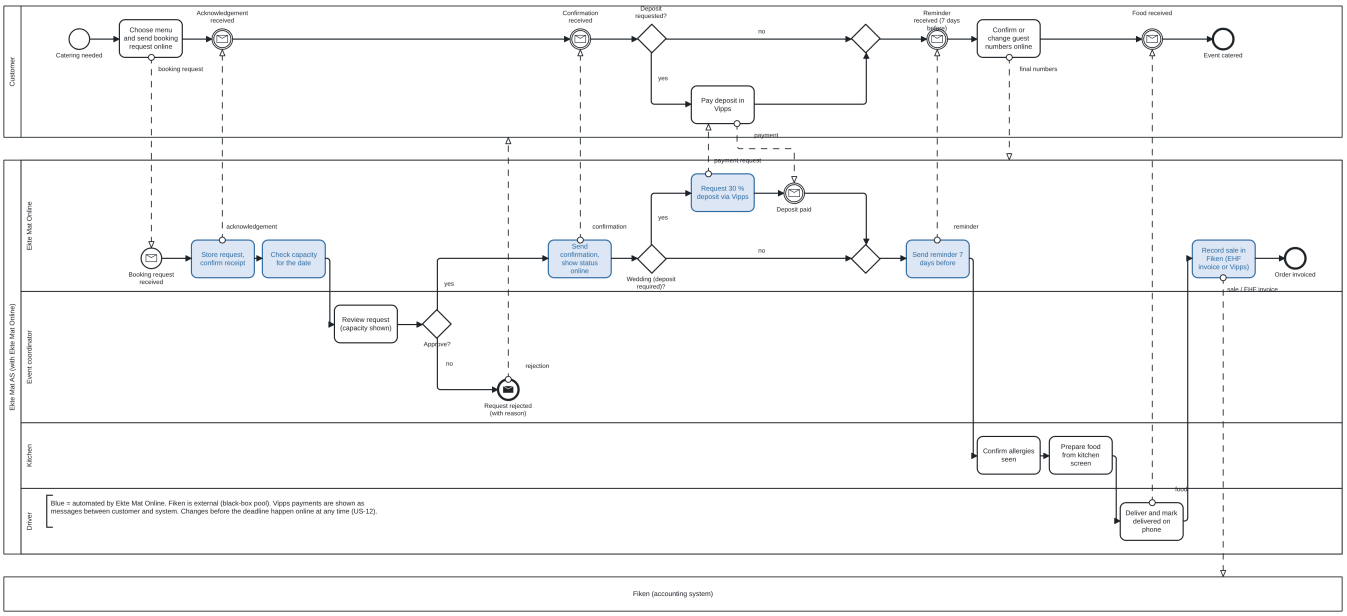
Used for the data model: the tables are a subset of it. Dish became `food_items`; Order, OrderLine (with the price frozen when the order is sent) and GuestGroup (allergies per group, never per named guest, US-40) became the order tables; OrderQuestion became `questions`. Six of its seven EventType values became the menu's events.

State machine for the order status



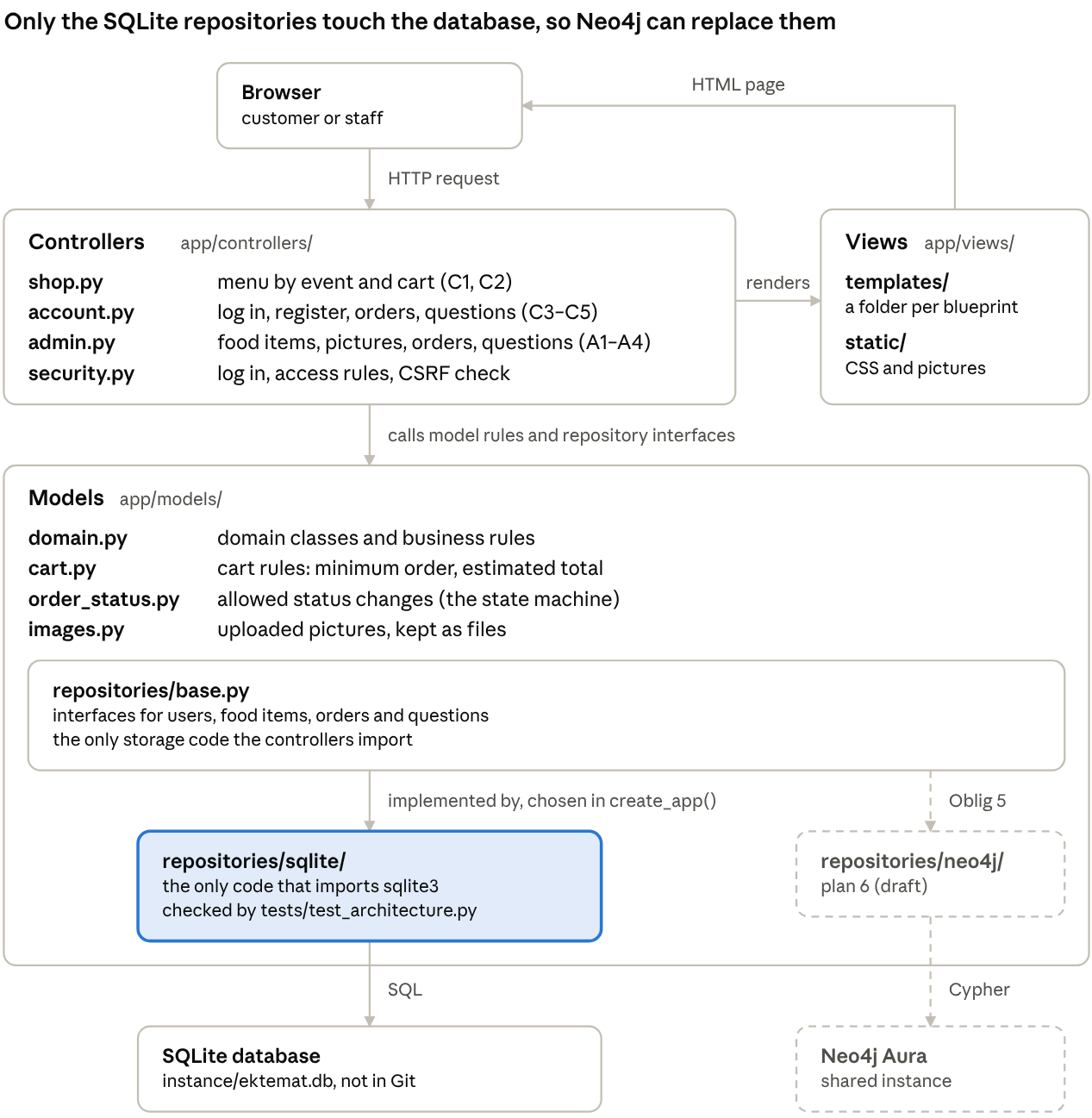
Used for the order rules. Its transitions became one table in `app/models/order_status.py`: submitted → approved, rejected or cancelled; approved → in preparation or cancelled; in preparation → delivered. The staff order page shows one button per allowed transition, and every other change is refused. States that Oblig 4 does not name (awaiting company approval, out for delivery, invoiced) were left out.

Business process (BPMN, to-be)



Used to check the flow end to end: the customer chooses a menu and sends a booking request online, and the coordinator approves or rejects it. Oblig 4 has no e-mail, so the process's acknowledgement and confirmation became the order page and its status, which the customer follows under My orders.

MVC architecture



MVC layers · the repository boundary

A request goes from the browser to a controller, which calls the model and renders a view; only `repositories/sqlite/` talks to the database. `tests/test_architecture.py` fails if `sqlite3` is imported anywhere else or if a controller imports the SQLite code, so Neo4j can replace SQLite in Oblig 5 with a second package and a change in `create_app()` only (plan 6).

Rules from the lab's data analysis

Our lab analysis of 820 rows of historical orders (40 with errors) gave ten rules. Each is enforced by the form, the database, or both.

Rule	Problem in the old data	Enforced by
V1	The same order entered twice	A unique order number; a hidden form key gives one order per click
V2	Channel spelled in different ways	Not stored: every order comes from the website
V3	A status typo	The app sets the status; the database accepts only the six statuses
V4	Customer type missing	The event type is a required field; `NOT NULL` in the database
V5	0 guests or fewer	A form rule and a database `CHECK`
V6	A guest count above the kitchen's daily capacity (5,000)	At most 400 guests: a form rule and a database `CHECK`
V7	Price per guest 0	Today's price is copied into each order line
V8	Total not equal to guests × price	No stored total: always the sum of the lines
V9	Event date before the order date	A form rule
V10	Postcode not four digits	A separate postcode field, a form rule and a database `CHECK`

Non-functional requirements

NFR-14 (a non-technical staff member adds a dish in at most 3 minutes) was timed by hand: about 1.5 minutes. NFR-19 became the dashboard's monthly share of requests answered within 24 hours, so NFR-05's 95 % target can be checked at all. NFR-01 (pages in at most 2 s over 4G) was not tested on a real 4G connection. On a local server with the seed data, the menu loaded in 26–140 ms in a fresh browser (eight loads). Real photos are larger than the seed's placeholder pictures (up to 1200 × 1200 px), so a test on a phone over 4G is still needed.

Image — Admin dashboard

Dashboard (NFR-19): orders and questions waiting, and the share of requests answered within 24 hours per month.

4. Did design help?

Yes. Design work decided much of what Claude Code built, and it caught problems before they became code. The user stories gave every plan its acceptance tests, so "done" was defined before any code existed. The order state machine became a table of allowed changes, so the staff page can never offer an impossible jump such as Submitted → Delivered. Our analysis of 820 old orders gave ten concrete rules: V1 (orders entered twice) led to protection against double clicks, and V10 (wrong postcodes) split the address into three fields. Reading US-47 closely showed, before ordering was built, that logging in would have emptied the customer's cart.

Design knowledge also let us steer Claude Code and judge its work. Every larger feature started as a written plan that we approved before any code was written, and each decision in a plan pointed to a requirement we knew (the 400-guest limit, for example, comes from V6), so we could judge it ourselves. Each plan's "What changed" section records where the agent was wrong and how it was caught: tests that checked the wrong text, a bug that only appeared on Windows and that a teammate found, and layout problems that only a browser check showed. Without the requirements and the models, we could not have told whether its work was right.